HEXADIGITALL BLUEPRINT

ECOSYSTEM ARCHITECTURE & PUBLICATION INTEGRATION SPECIFICATION

PROJECT NAME:	Hexadigitall Project Engine (hexadigitall.com)	**DOCUMENT REF:**	HEXA-BLU-2026-V1
AUTHOR IMPRINT:	FVMMD (Penname Workspace)	**COMPILED DATE:**	May 22, 2026
CORE TECH:	Next.js 15 (App Router), React 19, Sanity CMS, Paystack API	**TARGET DEPLOY:**	Vercel Edge Network

1. EXECUTIVE ARCHITECTURAL STRATEGY

The Hexadigitall ecosystem is designed as a unified, high-performance publishing engine optimized for literary, research, and interactive manual distribution. This blueprint maps the structural translation of physical raw content into digital architectures.

The platform manages digital content distribution with an infrastructure optimized for speed, programmatic access controls, and transparent checkout fulfillment paths. By separating high-scale dynamic components from server-side static generation, the engine ensures millisecond lookup responses even under intense concurrency spikes on the `hexadigitall.com` domain.

> **TARGET OBJECTIVE: SECTION C APPENDIX MATRIX**
>
> This model focuses on launching the initial literary work **"Love is Nothing. Love Comes From Everything Long Lasting"**, standardizing how end-matter resource elements (tracking sheets, diagnostic tools, and roadmap sheets) are stored, processed, searched, and securely served to verified book owners.

2. COMPLETE TECHNICAL STACK ARRAY

To preserve extreme visual fidelity and execution speeds, the architecture enforces a modern stack boundary with clear component responsibilities:

TECHNOLOGY TIER	ENGINE SUBSYSTEM COMPONENT	ARCHITECTURAL OPERATIONAL PURPOSE
Next.js 15	App Router Engine (`/app`)	Provides high-performance, edge-rendered Server Components for immediate client painting.
React 19	Concurrent Mode Boundaries	Enables asynchronous hydration and lightweight state streams for interface search fields.
Sanity CMS	GROQ Database Engine	Acts as an immutable data ledger containing content definitions, file matrices, and author profiles.
Paystack API	Webhook Execution Node	Processes high-volume financial checks and securely updates the asset authorization list.
Tailwind CSS	Utility Design Compiler	Ensures clean UI generation without shipping unnecessary CSS bundles to mobile users.
TypeScript	Strict Type Inference	Enforces compile-time type boundaries across all database payloads and client forms.

3. PRODUCTION DIRECTORY MAPPING

All modular content expansions and assets must strictly align to the following standardized file layout structures within the repository:

```
hexadigitall-project/
├── .build-scripts/
│   └── migrate-services.js        # Data seeding & migration execution script
├── app/
│   ├── api/
│   │   ├── publications/
│   │   │   └── search/
│   │   │       └── route.ts       # Performance edge search algorithm route
│   │   └── webhooks/
│   │       └── paystack/
│   │           └── route.ts       # Secure clearing automation endpoint
│   └── publications/
│       ├── page.tsx               # Base library and digital catalog entry point
│       └── [slug]/
│           ├── page.tsx           # Dynamic digital manuscript hub view
│           └── resource-vault/
│               └── page.tsx       # Section C gated asset download delivery vault
├── components/
│   └── publications/
│       ├── SearchInterface.tsx    # Responsive React 19 inline client component
│       └── ResourceCard.tsx       # Reusable element layout configuration
└── sanity/
```

```
    ├── schema.ts                 # Unified CMS compiler mapping connection file
    └── schemas/
        ├── author.ts             # Author metadata profile schema
        ├── publication.ts        # Digital literary item manifest schema
        └── resourceMatrix.ts     # Section C appendix tool reference schema
```

4. DATABASE ENGINE DEFINITIONS (SANITY SCHEMAS)

The content layout relies on a normalized graph structure. To connect Section C elements to manuscripts, the database architecture declares three distinct document schemas:

4.1. The Resource Matrix Schema

Tracks explicit tool definitions, diagnostic charts, and workbooks found inside the appendix sections of books.

```
// Location: sanity/schemas/resourceMatrix.ts
import { defineType, defineField } from 'sanity';

export default defineType({
  name: 'resourceMatrix',
  title: 'Section C Resource Tracker Matrix',
  type: 'document',
  fields: [
    defineField({
      name: 'title',
      title: 'Resource/Matrix Identifier Name',
      type: 'string',
      validation: (Rule) => Rule.required(),
    }),
    defineField({
      name: 'matrixId',
      title: 'Unique Section Reference Code',
      type: 'string',
      description: 'e.g., FVMMD-LIN-M3',
      validation: (Rule) => Rule.required(),
    }),
    defineField({
      name: 'resourceType',
      title: 'Medium Classification',
      type: 'string',
      options: {
        list: [
          { title: 'Clean Tracking Template (Interactive)', value: 'template' },
          { title: 'Educational Roadmap Manual (PDF)', value: 'roadmap' },
          { title: 'Architecture Roundtable Workshop Entry', value: 'workshop' },
        ],
      },
      validation: (Rule) => Rule.required(),
    }),
    defineField({
```

```
      name: 'secureAssetUrl',
      title: 'Protected Distribution File Link',
      type: 'url',
    }),
    defineField({
      name: 'requiresVerification',
      title: 'Enforce Book Proof of Purchase Entry Gate',
      type: 'boolean',
      initialValue: true,
    }),
  ],
});
```

4.2. The Publication Registry Schema

Manages high-level book details, indexing structural relationships, pricing boundaries, and its associated resource maps.

```
// Location: sanity/schemas/publication.ts
import { defineType, defineField } from 'sanity';

export default defineType({
  name: 'publication',
  title: 'Publication Knowledge Graph Registry',
  type: 'document',
  fields: [
    defineField({
      name: 'title',
      title: 'Complete Literary Work Title',
      type: 'string',
      validation: (Rule) => Rule.required(),
    }),
    defineField({
      name: 'slug',
      title: 'URL Route Slug Token',
      type: 'slug',
      options: { source: 'title', maxLength: 96 },
      validation: (Rule) => Rule.required(),
    }),
    defineField({
      name: 'author',
      title: 'Author Profile Node',
      type: 'reference',
      to: [{ type: 'author' }],
      validation: (Rule) => Rule.required(),
    }),
    defineField({
      name: 'isbn',
      title: 'International Standard Book Number',
      type: 'string',
    }),
    defineField({
```

```
      name: 'description',
      title: 'Manuscript Abstract / Index Overview',
      type: 'text',
    }),
    defineField({
      name: 'price',
      title: 'Base Cleardown Price (NGN/USD Equivalent)',
      type: 'number',
      validation: (Rule) => Rule.required().min(0),
    }),
    defineField({
      name: 'embeddedResources',
      title: 'Section C Appendix Matrix List Links',
      type: 'array',
      of: [{ type: 'reference', to: [{ type: 'resourceMatrix' }] }],
    }),
  ],
});
```

5. DYNAMIC SEARCHING ALGORITHM ARCHITECTURE

To bypass heavy infrastructure search engines, a customized dynamic algorithmic route is built using Next.js Edge capabilities. The routine fetches a localized high-density GROQ map and measures term weights on clean string arrays using strict boundary regex matches.

```
// Location: app/api/publications/search/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { client } from '@/sanity/lib/client';

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = request.nextUrl;
    const rawQueryString = searchParams.get('q') || '';

    if (!rawQueryString.trim()) {
      return NextResponse.json({ success: true, results: [] });
    }

    const dataExtractQuery = `*[_type == "publication"] {
      _id,
      title,
      "slug": slug.current,
      description,
      "authorName": author->name,
      "resources": embeddedResources[]-> { matrixId, title, resourceType }
    }`;

    const lookupDataset = await client.fetch(dataExtractQuery);
    const cleaningRegex = /[^\w\s]/g;
    const normalizationTokens = rawQueryString.toLowerCase().replace(cleaningRegex, '').split(/
\s+/).filter(Boolean);
```

```
    const matchEvaluationEngine = lookupDataset.map((document: any) => {
      let documentAccumulatedScore = 0;
      const searchTargetString = [
        document.title,
        document.description || '',
        document.authorName,
        document.resources?.map((r: any) => `${r.matrixId} ${r.title}`).join(' ') || ''
      ].join(' ').toLowerCase().replace(cleaningRegex, '');

      normalizationTokens.forEach((token) => {
        if (searchTargetString.includes(token)) {
          const boundaryScan = new RegExp(`\\b${token}\\b`, 'i');
          documentAccumulatedScore += boundaryScan.test(searchTargetString) ? 10 : 3;
        }
      });

      return { node: document, calculatedMatchScore: documentAccumulatedScore };
    });

    const filteredRankedResults = matchEvaluationEngine
      .filter((candidate: any) => candidate.calculatedMatchScore > 0)
      .sort((alpha: any, beta: any) => beta.calculatedMatchScore - alpha.calculatedMatchScore)
      .map((entry: any) => entry.node);

    return NextResponse.json({ success: true, results: filteredRankedResults });
  } catch (err) {
    return NextResponse.json({ success: false, error: 'Internal Engine Compute Failure' },
{ status: 500 });
  }
}
```

6. PAYSTACK FINANCIAL FULFILLMENT PIPELINE

When an analytical book sale executes successfully on the front end, Paystack dispatches a secure cryptographic payload. The application parses the hash signature against local server secrets to instantly instantiate user authorization logs inside Sanity.

```
// Location: app/api/webhooks/paystack/route.ts
import { NextRequest, NextResponse } from 'next/server';
import crypto from 'crypto';
import { client } from '@/sanity/lib/client';

export async function POST(request: NextRequest) {
  try {
    const requestRawPayload = await request.text();
    const payloadHashSignature = request.headers.get('x-paystack-signature');
    const locallyCalculatedHash = crypto
      .createHmac('sha512', process.env.PAYSTACK_SECRET_KEY!)
      .update(requestRawPayload)
      .digest('hex');
```

```
    if (payloadHashSignature !== locallyCalculatedHash) {
      return NextResponse.json({ error: 'Signature Verification Failure Exception' }, { status:
401 });
    }

    const payloadEventData = JSON.parse(requestRawPayload);

    if (payloadEventData.event === 'charge.success') {
      const { metadata } = payloadEventData.data;
      const customerEmail = payloadEventData.data.customer.email;
      const targetedPublicationId = metadata.publicationId;

      await client.create({
        _type: 'accessGrantLedger',
        customerIdentityToken: customerEmail,
        purchasedPublicationNode: { _type: 'reference', _ref: targetedPublicationId },
        grantedSystemTimestamp: new Date().toISOString(),
        operationalLedgerState: 'active',
      });
    }

    return NextResponse.json({ received: true }, { status: 200 });
  } catch (fault) {
    return NextResponse.json({ error: 'Fulfillment Error Lifecycle Catch' }, { status: 500 });
  }
}
```

7. PROTECTED RESOURCE VAULT (NEXT.JS SERVER PAGE)

The runtime interface utilizes Next.js Server Components to pull structured information, mapping safe access pathways without exposing secure layout definitions on the browser tier.

```
// Location: app/publications/[slug]/resource-vault/page.tsx
import { notFound } from 'next/navigation';
import { client } from '@/sanity/lib/client';

export default async function ResourceVaultPage({ params }: { params: Promise<{ slug: string }
> }) {
  const { slug } = await params;

  const extractionQuery = `*[_type == "publication" && slug.current == $slug][0] {
    _id,
    title,
    "resources": embeddedResources[]-> { _id, title, matrixId, resourceType, secureAssetUrl }
  }`;

  const documentPayload = await client.fetch(extractionQuery, { slug });
  if (!documentPayload) notFound();
```

```
  return (
```

{DOCUMENTPAYLOAD.TITLE} — RESOURCE VAULT

```
        {documentPayload.resources?.map((res: any) => (
            {res.matrixId}
```

{res.title}

```
            Download Asset Matrix →
        ))}
  );
}
```